

|                                                                                            |                                                                        |                                |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------|
| <b>Université de Carthage</b><br><b>Ecole Nationale</b><br><b>d'Ingénieurs de Carthage</b> | <b>Atelier</b><br><b>Web service : <i>REST</i></b><br><b>2025-2026</b> | <b>3ING Génie Informatique</b> |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------|

## Objectifs

Le but de cet atelier est :

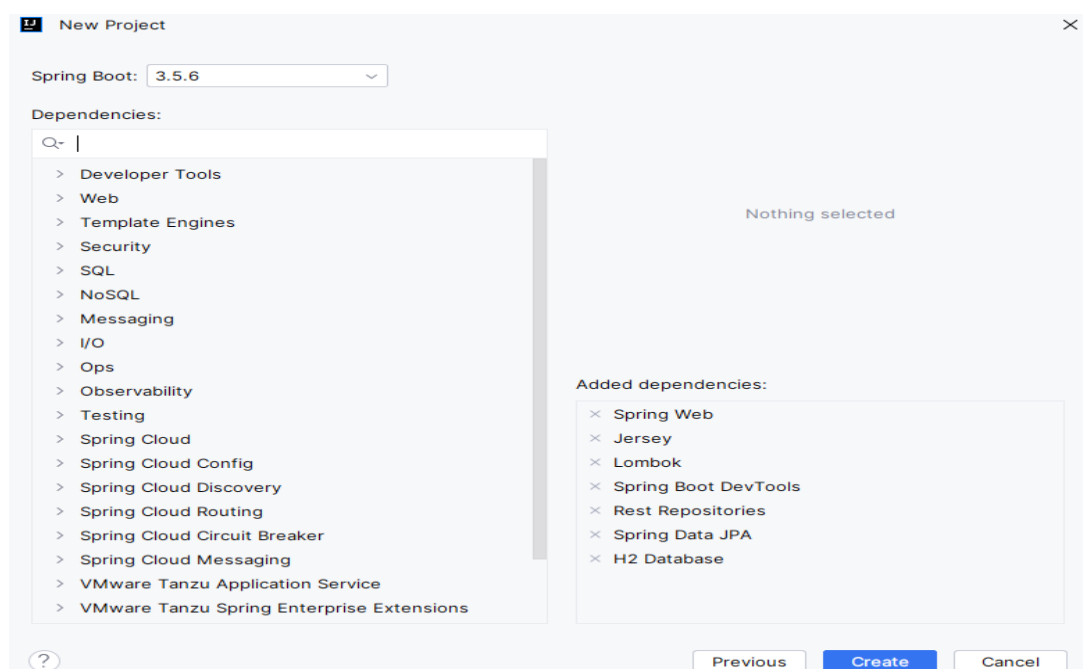
- La création d'un service web REST « **banqueWSREST** » de 2 façons différentes en utilisant JAX-RS avec l'implémentation de référence Jersey puis à l'aide de RESTController.
- Le test du service web et la visualisation des requêtes http avec SOAP UI ou POSTMAN

## Méthodologie de développement du service web

1. Seule l'approche Bottom/UP est disponible
2. Créer et annoter un POJO
3. Compiler, Déployer et Tester

## L'implémentation du service

1. Créez un projet SpringBoot à travers SpringInitializer que vous l'appeler **banqueWSREST**



2. La couche DAO de votre application est accessible via le lien suivant  
<https://classroom.google.com/c/ODA2ODg0MDIwMzk5/m/ODE2NDUyMjAzODM5/details>
3. Exécuter et consulter votre base de données de type H2 à travers ce lien : <http://localhost:8082/h2-console>
4. Utilisez cette couche DAO pour implémenter le service REST de deux manières : soit en utilisant Jersey, soit en utilisant un RESTController

### Partie A : REST API à l'aide de la servlet Jersey

1. Créez la classe de votre service REST dans le package « web » et nommez-la classe « CompteJAXRSAPI »
2. Utilisez l'implémentation du service du TP SOAP qui permet de
  - Convertir un montant du dinar en euro
  - Récupérer un compte client
  - Consulter la liste de comptes disponibles.
3. Utiliser les annotations JAX-RS pour annoter la classe du service
4. Déployez JERSEY
  - **Dans le cadre d'un projet Maven simple** : il suffit d'ajouter manuellement certaines dépendances pour importer les bibliothèques nécessaires au déploiement de Jersey, puis de déclarer les servlets dans le fichier `web.xml`.
  - **Avec un projet Spring Boot** : il n'est pas nécessaire de créer un fichier `web.xml` pour déployer les servlets. Il suffit de créer une classe de configuration dans le package de l'application, que vous pouvez nommer `MyConfig`.
  - **Configurer la classe** : ajoutez l'annotation `@Configuration` pour indiquer à Spring qu'il s'agit d'une classe de configuration, puis créez une méthode pour enregistrer vos endpoints Jersey.

```
@Configuration
public class MyConfig {
    @Bean
    public ResourceConfig resourceConfig(){
        ResourceConfig jerseyServlet=new ResourceConfig();
        jerseyServlet.register(CompteJAXRSAPI.class);
        return jerseyServlet;
    }
}
```

5. Configurer votre application :

```
spring.datasource.url=jdbc:h2:mem:banque
spring.h2.console.enabled=true
server.port=8089
```

6. Tester le service via l'URL suivante : <http://localhost:8089/banque/comptes>
7. Tester ces opérations en utilisant POSTMAN ou SOAPUI

**Remarque :** Vous pouvez remarquer que les résultats obtenus sont sérialisés en JSON. Votre servlet Jersey utilise la bibliothèque Jackson pour cette sérialisation.

Si vous souhaitez sérialiser les données en XML, il faut d'abord ajouter l'annotation :

1. `@Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})` ainsi que la dépendance JAXB.
2. La classe que vous souhaitez sérialiser doit utiliser une annotation JAXB, notamment `@XmlRootElement`, dans notre cas « Compte »

Ajouter ces dépendances relatives à la bibliothèque JAXB, permettant à Spring Boot de générer automatiquement la réponse au format **XML**, en plus du JSON par défaut.

```
<!-- Jackson XML pour sérialisation XML -->
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
</dependency>

<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
  <version>4.0.0</version>
</dependency>

<!-- Jersey JAXB providers for XML (MessageBodyReader/Writer) -->
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-jaxb</artifactId>
  <version>${jersey.version}</version>
</dependency>
```

## Partie B : REST API à l'aide de SPRING MVC

1. Désactivez la configuration de la servlet Jersey en mettant l'annotation en commentaire `//@Bean`, afin d'éviter que le composant soit exécuté et chargé en mémoire au démarrage de l'application.

2. Copiez le contenu de la classe CompteJAXRSAPI et collez-le dans une nouvelle classe que vous créez sous le nom CompteRestController, puisque la structure reste la même.
3. Ajoutez les annotations de Spring MVC pour créer l'API REST
4. Tester votre Web Service via SOAP UI ou POSTMAN
5. Sérialiser la réponse selon le format demandé par l'utilisateur

### **Configurer le format de sortie dans le contrôleur**

Dans ta méthode de contrôleur, utilise produces pour indiquer les formats supportés par exemple :

```
@GetMapping(value = "/comptes", produces =  
{MediaType.APPLICATION_JSON_VALUE, MediaType.APPLICATION_XML_VALUE})  
public List <Compte> compteList(){  
  
    return compteRepository.findAll();  
}
```

6. Retester le web service avec le format XML en indiquant dans le header Accept: application/xml